Exercício 4.4: Desafio: canary manual medido, e o que o KEDA faria diferente

Aluno: Luiz Brito · **Data:** 19/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster kind v0.32.0 com 3 nós, Kubernetes v1.36.1, Calico v3.33.0, namespace `store-dev`.
- `store-api:1.0` e `store-api:2.0` geradas do mesmo código (`app/store-api`) com `--build-arg APP_VERSION`. O `/info` passou a retornar o campo `versao` e foi criado `/versao`, que devolve só a versão.
- KEDA 2.21.0 instalado com o comando do enunciado (`helm install keda kedacore/keda -n keda --create-namespace`).

Manifestos

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-api-stable
  namespace: store-dev
spec:
  replicas: 9
  selector:
    matchLabels:
      app: store-api
      version: stable
  template:
    metadata:
      labels:
        app: store-api
        version: stable
    spec:
      containers:
        - name: store-api
          image: store-api:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8000
          readinessProbe:
            httpGet:
              path: /ready
              port: http
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              memory: 256Mi
```

`02-canary.yaml` é igual trocando `stable` por `canary`, `replicas: 1` e a imagem `store-api:2.0`.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: store-api
  namespace: store-dev
spec:
  selector:
    app: store-api
  ports:
    - port: 80
      targetPort: http
```

Medição 9:1

```
$ kubectl -n store-dev get endpointslice -l kubernetes.io/service-name=store-api -o wide
NAME              ADDRESSTYPE   PORTS   ENDPOINTS                                               AGE
store-api-xgvt4   IPv4          8000    192.168.29.217,192.168.163.74,192.168.13.22 + 7 more...   26s

$ kubectl -n store-dev run cliente --rm -i --image=curlimages/curl:8.10.1 --restart=Never -- sh -c \
    'for i in $(seq 1 500); do curl -s http://store-api/versao; echo; done' | sort | uniq -c | sort -
rn
    447 {"versao":"1.0"}
     50 {"versao":"2.0"}
```

O loop chama `/versao` em vez de `/info` porque o `/info` traz também o nome do Pod e o `uniq -c` contaria por Pod, não por versão. Das 500 requisições, 497 voltaram com corpo: as 3 primeiras se perdem enquanto o `kubectl run -i` ainda está anexando ao Pod.

Resultado: canary com 50 de 497 = 10,1% (teórico 10%).

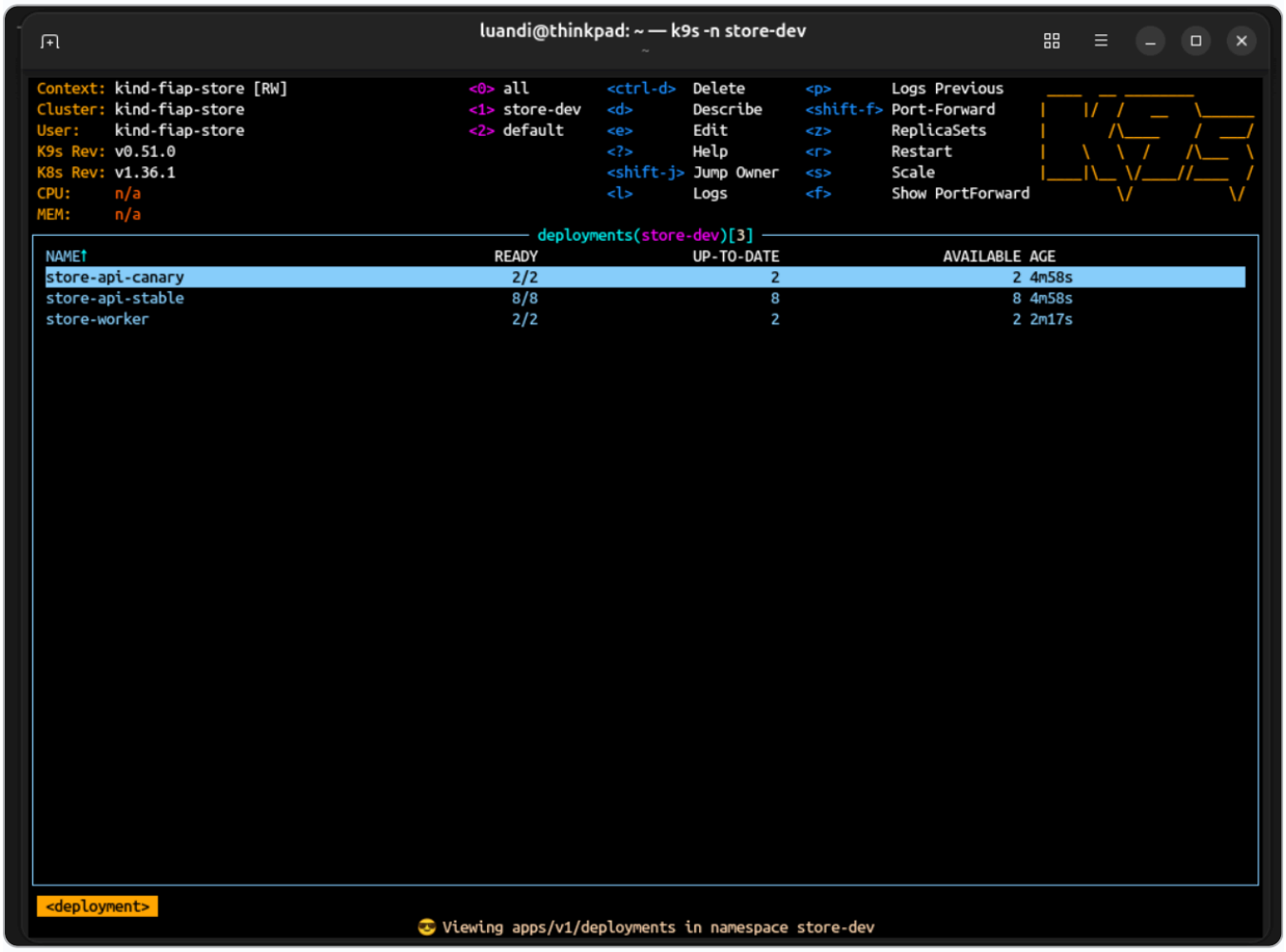
Medição 8:2

```
$ kubectl -n store-dev scale deploy/store-api-stable --replicas=8
deployment.apps/store-api-stable scaled
$ kubectl -n store-dev scale deploy/store-api-canary --replicas=2
deployment.apps/store-api-canary scaled

$ kubectl -n store-dev get deploy
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
store-api-canary   2/2     2            2           69s
store-api-stable   8/8     8            8           69s

$ (mesmo loop de 500 requisições)
    410 {"versao":"1.0"}
     87 {"versao":"2.0"}
```

Resultado: canary com 87 de 497 = 17,5% (teórico 20%).



Por que o medido difere do teórico

- **kube-proxy.** No modo iptables (padrão do kind), cada conexão nova cai num endpoint escolhido por regra `statistic mode random`: é sorteio, não round-robin. Com 500 amostras e p = 0,2 o desvio padrão é cerca de 9 requisições (1,8 ponto percentual), então 17,5% está dentro do esperado para um sorteio.
- **Keep-alive.** Cada `curl` do loop abre uma conexão nova, por isso cada requisição é um sorteio independente. Um cliente que reutiliza a conexão (keep-alive) fica preso ao mesmo Pod e distorce tudo.
- **sessionAffinity.** O Service usa o padrão `None`. Com `ClientIP`, todas as 500 requisições do mesmo Pod cliente iriam para o mesmo endpoint e a medição daria 0% ou 100%.

KEDA: ScaledObject com trigger cron

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-worker
  namespace: store-dev
spec:
  replicas: 0
  selector:
    matchLabels:
      app: store-worker
  template:
    metadata:
      labels:
        app: store-worker
    spec:
      containers:
        - name: store-worker
          image: busybox:1.36
          command: ["sh", "-c", "while true; do echo processando pedidos; sleep 30; done"]
          resources:
            requests:
              cpu: 10m
              memory: 16Mi
```

```yaml
# Janela curta para a demonstracao: ativo do minuto 0 ao 5 de cada dezena
# (x0 a x5). Fora dela o store-worker vai a 0 apos o cooldownPeriod.
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: store-worker
  namespace: store-dev
spec:
  scaleTargetRef:
    name: store-worker
  minReplicaCount: 0
  maxReplicaCount: 10
  cooldownPeriod: 30
  triggers:
    - type: cron
      metadata:
        timezone: America/Sao_Paulo
        start: "*/10 * * * *"
        end: "5-59/10 * * * *"
        desiredReplicas: "2"
```

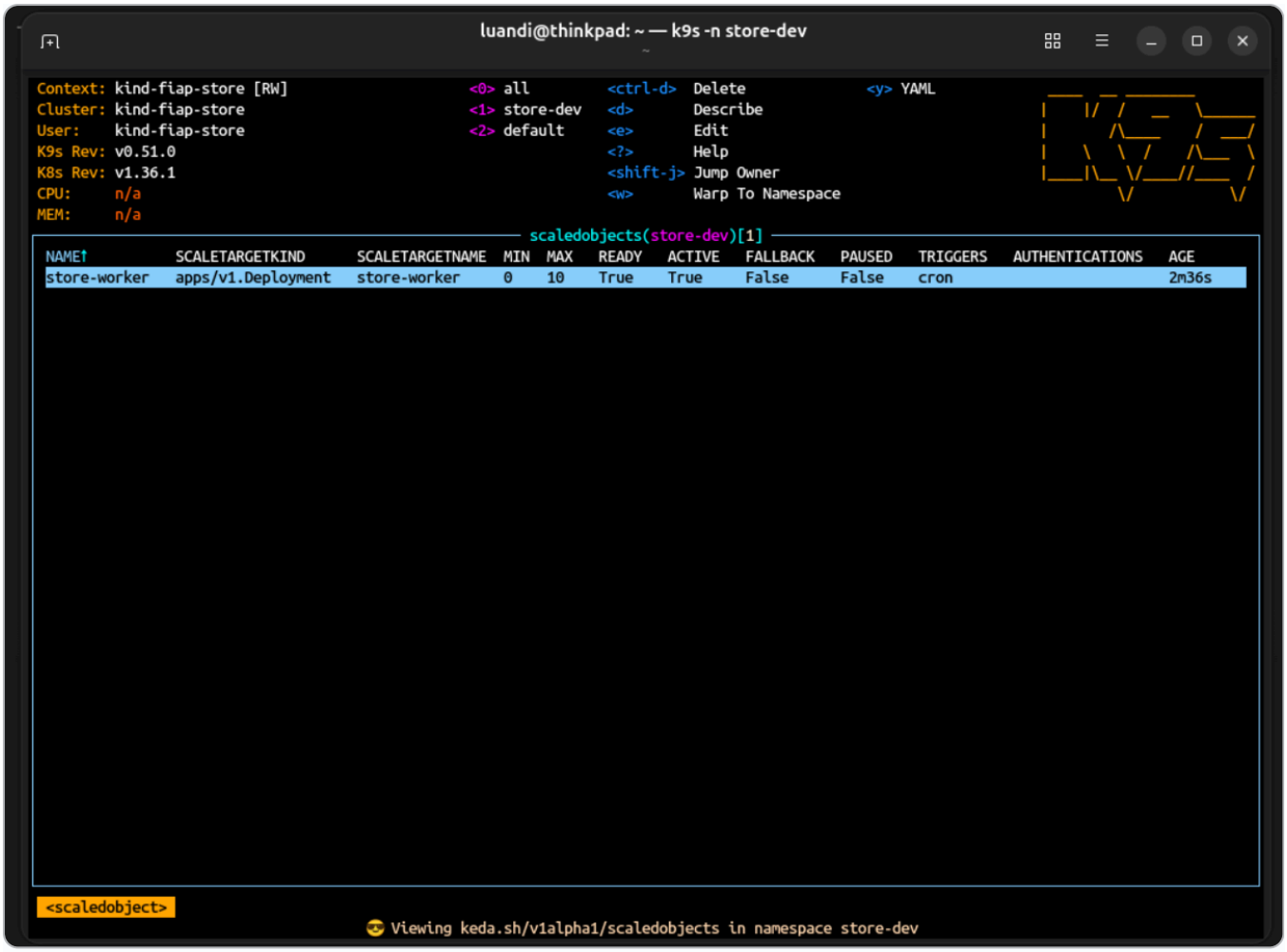
A janela foi encurtada para 5 minutos a cada 10 para dar para ver as duas fases. `cooldownPeriod: 30` reduz a espera de 300s (padrão) para voltar a zero.

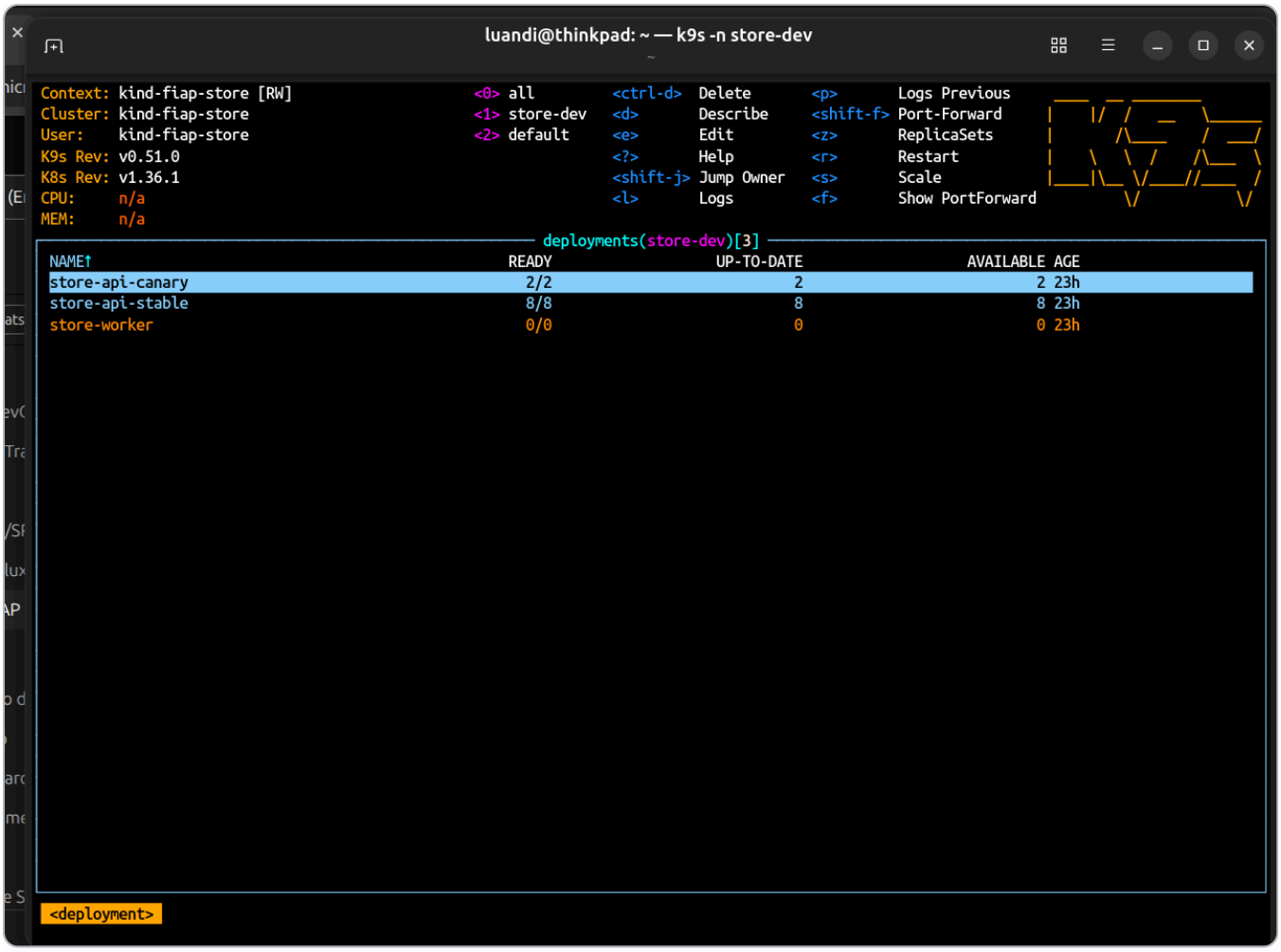
```
$ date +%T; kubectl -n store-dev get scaledobject,deploy/store-worker
11:19:57
NAME           MIN   MAX   READY   ACTIVE
store-worker   0     10    True    Unknown
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
store-worker   0/0     0            0           22s

$ date +%T; kubectl -n store-dev get scaledobject,deploy/store-worker
11:20:42
NAME           MIN   MAX   READY   ACTIVE
store-worker   0     10    True    True
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
store-worker   2/2     2            2           67s

$ date +%T; kubectl -n store-dev get scaledobject,deploy/store-worker
11:25:08
NAME           MIN   MAX   READY   ACTIVE
store-worker   0     10    True    False
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
store-worker   0/0     0            0           5m34s
```

Fora da janela (11:19) o worker está `0/0`; dentro dela (11:20) sobe para 2; depois do fim (11:25) volta a `0/0` com `ACTIVE False`.


luandi@thinkpad: ~ — k9s -n store-dev
Context: kind-fiap-store [RW]
Cluster: kind-fiap-store
User: kind-fiap-store
K9s Rev: v0.51.0
K8s Rev: v1.36.1
CPU: n/a
MEM: n/a
<0> all
<1> store-dev
<2> default
<ctrl-d> Delete
<d> Describe
<e> Edit
<?> Help
<shift-j> Jump Owner
<w> Warp To Namespace
<y> YAML
scaledobjects(store-dev)[1]
NAME↑ SCALETARGETKIND SCALETARGETNAME MIN MAX READY ACTIVE FALLBACK PAUSED TRIGGERS AUTHENTICATIONS AGE
store-worker apps/v1.Deployment store-worker 0 10 True True False False cron 2m36s
<scaledobject>
Viewing keda.sh/v1alpha1/scaledobjects in namespace store-dev



Comparativo: canary por réplicas x KEDA

(i) Granularidade de 1%. Por contagem de réplicas o percentual é `canary / total`. Para 1% são 99 stable + 1 canary: 100 Pods só para controlar o peso, mesmo que 4 dessem conta da carga. O custo é capacidade ociosa (CPU e memória reservadas por `requests`), mais nós e mais tempo de rollout. Na prática fica preso a degraus de 10% ou 20% com poucas réplicas, e o resultado ainda varia: medi 10,1% em 9:1 e 17,5% em 8:2.

(ii) Peso no Ingress. Com roteamento por peso (anotações `canary-weight` do ingress-nginx, Gateway API `HTTPRoute` com `weight` ou um service mesh) o proxy decide o destino de cada requisição. A proporção deixa de depender do número de Pods: 1% com 1 Pod canary e 2 stable. Também dá para rotear por header ou cookie (testers internos) e o escalonamento de cada versão fica livre, por exemplo com HPA.

(iii) Scale-to-zero. O `store-worker` processa pedidos em lote: não tem cliente esperando resposta, então o atraso de subir um Pod (cold start de segundos) é irrelevante e existe um gatilho natural (fila, cron). A `store-api` atende requisições síncronas do usuário; com 0 réplicas a primeira requisição cairia em erro ou esperaria o

Pod subir, o que quebra a experiência e a readiness. Ela precisa de mínimo maior que zero.

(iv) Custo mensal, preço Fargate us-east-1 (página oficial: US$ 0,000011244 por vCPU-segundo e US$ 0,000001235 por GB-segundo, ou seja US$ 0,04048/vCPU-h e US$ 0,00445/GB-h). Pod do worker com 0,25 vCPU e 0,5 GB = US$ 0,01234/h. Duas réplicas:

Abordagem	Horas/mês	Custo/mês
Worker sempre ligado (Deployment comum)	2 × 720 = 1.440	US$ 17,77
Worker com KEDA, 2 h/dia	2 × 60 = 120	US$ 1,48

Economia de cerca de 92%. O control plane do EKS (US$ 0,10/h, página oficial, US$ 72/mês) é igual nos dois cenários, e o próprio KEDA roda nos nós do cluster. O canary por réplicas tem o efeito contrário: para 1%, 100 Pods iguais a esse custariam cerca de US$ 888/mês contra US$ 26,66/mês de 3 Pods com peso no Ingress.